

## SYSTEM DESIGN SPECIFICATION

# Real-Time Fraud & Anomaly Detection Platform

A streaming, explainable, graph-aware fraud detection system with human-in-the-loop review and full MLOps + LLMOps observability.

KAFKA

DATABRICKS

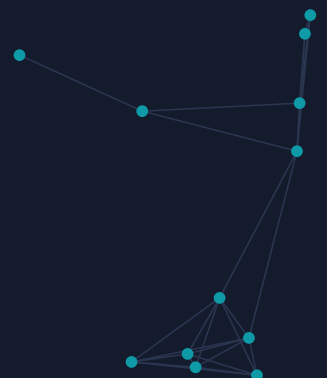
MONGODB

STREAMLIT

LANGCHAIN

LANGSMITH

Prepared for: Mohammed Shadab Ameen · Edison Internship, GE HealthCare  
Version 1.0 · Portfolio / System-Design Review Edition



00

Contents

| #  | Section                       | What it covers                                                 |
|----|-------------------------------|----------------------------------------------------------------|
| 01 | Executive Summary             | Problem, value, and what makes it stand out                    |
| 02 | Objectives & Requirements     | Functional + non-functional (latency, throughput, reliability) |
| 03 | System Architecture           | Hot path / cold path / analyst-assist — full diagram           |
| 04 | Component Deep-Dive           | Producers, Kafka, Databricks, MongoDB, Streamlit               |
| 05 | Machine Learning Layer        | Models, MLflow, thresholds, imbalance strategy                 |
| 06 | Graph-Based Ring Detection    | Neptune/Neo4j, community detection, graph features             |
| 07 | LLM Analyst-Assist Layer      | LangChain chains, RAG, investigation agent                     |
| 08 | LLMOps with LangSmith         | Tracing, evaluation, monitoring, feedback                      |
| 09 | Data Engineering Backbone     | Delta medallion, Feature Store, orchestration, quality         |
| 10 | End-to-End Data Flow          | The 8-step story, step by step                                 |
| 11 | Data Models & Schemas         | Kafka payloads + MongoDB collections                           |
| 12 | Trade-offs & Design Decisions | What to defend in a review                                     |
| 13 | Security & Reliability        | PII, adversarial, DLQ, exactly-once                            |
| 14 | Build Plan & Milestones       | Phased 4-week (and compressed 2-week) plan                     |
| 15 | Tech Stack & Success Metrics  | Full stack table + KPIs                                        |

## 01

## Executive Summary

This document specifies a **real-time fraud and anomaly detection platform** that ingests a high-velocity transaction stream, scores every event in near real-time, flags suspicious activity for human review, and continuously improves itself from analyst feedback. It is engineered as a portfolio-grade system-design showcase: it demonstrates streaming architecture, applied machine learning, graph analytics, retrieval-augmented LLM tooling, and disciplined MLOps/LLMOps.

The design is deliberately split into a **hot path** that makes fast, deterministic scoring decisions and a **cold path** that stores everything durably and periodically retrains the model. Large-language-model reasoning is kept strictly **off the hot path** — it powers an on-demand "AI fraud analyst" that explains why a transaction was flagged, grounded in real feature attributions and precedent cases.

### What makes this project stand out

- **Hybrid detection:** a per-transaction ML model *plus* a graph layer that catches fraud *rings* a point-model cannot see.
- **Explainable by design:** SHAP feature attributions feed a LangChain summariser, so every alert ships with a plain-English, cited rationale.
- **Two observability planes:** MLflow watches the scoring model; LangSmith watches the LLM assistant — a mature, production-credible story.
- **Closed feedback loop:** analyst decisions retrain the model and grow an LLM evaluation set, so both systems get better over time.
- **Cost- and latency-aware:** autoscaling streaming clusters, on-demand LLM calls, and a live cost/SLA dashboard.

#### The one-line pitch

"An adaptive, explainable, graph-aware fraud-detection system with proper MLOps and LLMOps" — not just a streaming pipeline.

02

Objectives & Requirements

Functional requirements

- Ingest a continuous stream of transaction events without loss.
- Enrich each event with rolling behavioural features and graph context, then score it for fraud risk.
- Assign a risk band (low / medium / high) and persist decisions to a low-latency serving store.
- Surface flagged transactions on a live dashboard with a human review workflow.
- Generate a grounded, natural-language explanation for any flagged case on demand.
- Retrain and promote improved models from accumulated analyst labels — without redeploying the stream.

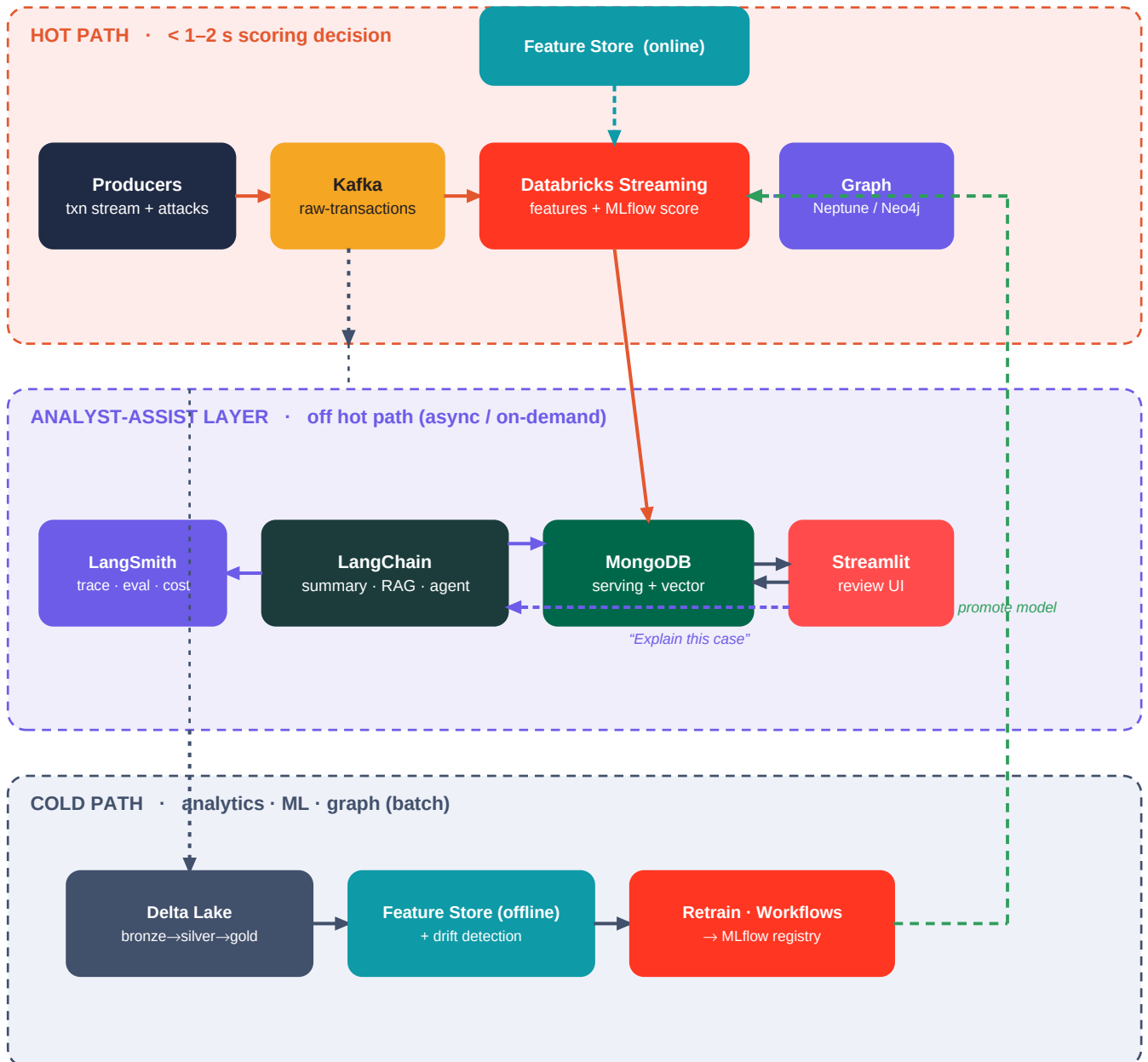
Non-functional requirements

| Attribute    | Target                         | How the design meets it                                         |
|--------------|--------------------------------|-----------------------------------------------------------------|
| Latency      | < 1–2 s per event (hot path)   | Stateful streaming; model held in-memory; LLM kept off hot path |
| Throughput   | 5k–50k events/sec, bursty      | Kafka buffering + partitioning by user_id; autoscaling clusters |
| Reliability  | No lost transactions           | At-least-once ingest + idempotent upserts keyed on txn_id       |
| Auditability | Every decision reproducible    | Raw + enriched history in Delta; full model/feature lineage     |
| Adaptability | Models never go stale silently | Drift detection triggers retraining; challenger shadow deploys  |
| Cost         | Predictable, observable        | Autoscaling job clusters; on-demand LLM; live cost dashboard    |

## 03

## System Architecture

The platform is organised into three planes. The **hot path** (top) turns raw events into scored decisions in under two seconds. The **analyst-assist layer** (middle) runs LLM reasoning on demand, off the critical path. The **cold path** (bottom) is the durable analytics and retraining backbone. Green dashed arrows show the feedback loops that make the system adaptive.



**Reading the diagram:** solid coral arrows are the low-latency scoring path; purple dashed arrows are on-demand LLM/graph lookups; grey dashed arrows feed the cold path; the green dashed arrow is the model-promotion loop back into the live stream.

#### Design principle #1 — separate hot and cold

The single most important idea to articulate in a review: fast decisions and heavy learning have different SLAs, so they run on different paths against different stores (MongoDB for serving, Delta Lake as the source of truth for training).

## 04

## Component Deep-Dive

### Producers — the event source

A transaction service (or a synthetic generator for the demo) emits events with fields such as `txn_id`, `user_id`, `amount`, `merchant`, `merchant_category`, `geo`, `device_id`, `ip`, `timestamp`. The generator injects a controllable fraud ratio (~2%) with realistic patterns — velocity spikes, geo-impossible travel, card-testing bursts — so the dashboard always has something meaningful to show. Seed realism with the Kaggle Credit-Card-Fraud dataset or the PaySim mobile-money simulation.

### Apache Kafka — ingestion & buffering

- Topics: `raw-transactions` (input), optionally `scored-transactions` and `alerts`.
- **Partition by `user_id`** so a user's events stay ordered on one partition — essential for velocity features.
- Acts as a shock absorber: decouples bursty producers from processing and provides replayability.
- Design talking points: partition count vs consumer parallelism, `acks=all` durability, offset management, a dead-letter topic for poison messages.

### Databricks (Structured Streaming) — the brain

Reads Kafka via the native source and performs **stateful stream processing** with watermarking to compute rolling features per user: transaction counts over 1/5/60-minute windows, rolling mean and standard deviation of amount, distinct merchants and geos, and time since last transaction. It pulls the current model from the MLflow registry, scores each event, then writes decisions to MongoDB (serving) and appends raw + scored rows to Delta Lake (history).

- Design talking points: exactly-once vs at-least-once semantics, watermarking & late data, checkpointing for fault tolerance, state TTL to bound memory.

### MongoDB — the serving layer

Enriched transactions are naturally document-shaped (nested feature objects, variable merchant metadata) and the dashboard needs fast reads — a strong, defensible fit for MongoDB.

| Collection                        | Purpose                                         | Key / index                                                                                        |
|-----------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>flagged_transactions</code> | Full enriched doc + score + risk band           | <code>_id = txn_id</code> ; idx on <code>user_id</code> , <code>risk_band</code> , <code>ts</code> |
| <code>user_rolling_state</code>   | Latest aggregates per user (idempotent upsert)  | <code>_id = user_id</code>                                                                         |
| <code>alerts</code>               | Open/closed alert lifecycle for review          | idx on <code>status</code> , <code>priority</code>                                                 |
| <code>case_embeddings</code>      | Vectors of resolved cases (Atlas Vector Search) | vector index (cosine)                                                                              |

### Streamlit — UI & operations console

- Live alerts feed (auto-refresh) with geo-map and score-distribution charts.
- Review panel: analyst marks fraud / not-fraud, writing a label back to Mongo that feeds retraining.
- "Explain this case" button that invokes the LangChain assistant on demand.
- KPIs: flag rate, alerts/hour, model version in use, and live p50/p95/p99 scoring latency.

## 05

## Machine Learning Layer

Fraud labels are scarce and highly imbalanced, so the model strategy evolves in stages rather than assuming clean supervision from day one.

| Stage     | Model                          | When / why                                    |
|-----------|--------------------------------|-----------------------------------------------|
| Bootstrap | Isolation Forest / Autoencoder | Unsupervised; works before you have labels    |
| Mature    | XGBoost / LightGBM             | Supervised, once review labels accumulate     |
| Augmented | + graph features (see §06)     | Ring signals lift recall on coordinated fraud |

### MLflow — experiment tracking & registry

- Tracks experiments, parameters, and metrics; versions every model.
- The stream loads the model tagged **Production** via the registry — promotion happens without a redeploy.
- Enables challenger / shadow evaluation before a model is promoted.

### Thresholds & imbalance strategy

- Map raw score → risk band: **high** = auto-flag; **medium** = queue for human review; **low** = pass.
- Optimise for **recall on the fraud class** while watching false-positive fatigue for analysts.
- Rank alerts by **expected loss** (score × amount) so analysts work the highest-value cases first.

#### Design principle #2 — plan for drift

Fraud is adversarial: attackers adapt, so models decay fast. Monitor feature drift (population stability index) and score-distribution drift, and trigger retraining when thresholds are crossed — don't rely on a nightly cron alone.

## 06

## Graph-Based Ring Detection

Fraud is rarely a lone transaction — it is organised into **rings**: accounts sharing devices, IPs, or billing addresses, and money mules moving funds in chains. A per-transaction model is blind to these relationships. Adding a graph layer (AWS Neptune or Neo4j) makes them visible.

### Model & techniques

- Model relationships: User → Device → Merchant → Card / IP → Address.
- Run **community detection** and **connected-components** to surface suspicious clusters.
- Compute **graph features** — e.g. "how many flagged users share this device?" — and feed them back into the scoring model as additional inputs.

### Why this is the standout addition

It converts a standard streaming project into a hybrid graph + ML system — the single most impressive differentiator, and a natural showcase for knowledge-graph expertise.

## 07

## LLM Analyst-Assist Layer (LangChain)

### Golden rule — keep LLMs off the hot path

A per-transaction scoring decision must never block on an LLM call. LangChain and LangSmith live in the analyst-assist layer, triggered AFTER a transaction is flagged, on demand.

## Three concrete LangChain uses

### 1 · Case-summary chain

Takes the flagged transaction + its SHAP feature attributions + the user's recent history and produces a plain-English rationale — e.g. "amount 8× the user's average, a new geo, and 4 transactions in 60 seconds." This is the LLM realisation of the explainability requirement.

### 2 · RAG over fraud playbooks & past cases

A retrieval chain queries **MongoDB Atlas Vector Search** over embeddings of resolved fraud cases and internal fraud-policy documents, so each summary is grounded in precedent: "this pattern matches three confirmed card-testing cases from last month."

### 3 · Investigation agent (advanced)

A LangChain agent equipped with tools — `query_graph(user_id)` (Neptune/Neo4j ring lookup), `get_user_history(user_id)` (Mongo), and `similar_cases(embedding)` (vector search). The analyst asks "is this part of a ring?" and the agent orchestrates the lookups — tying graph + retrieval + LLM into one showpiece.

### Two things that stop it looking bolted-on

(1) Make the LLM call explicitly async / button-triggered and say so in the review. (2) Always ground the chain in real SHAP features + retrieved cases — an ungrounded summary adds nothing; a grounded, cited one is the whole point.

08

LLMOps with LangSmith

LangSmith provides observability and evaluation for the LLM layer — the LLMOps counterpart to MLflow.

| Capability | What it does here                                                             | Why it matters                                    |
|------------|-------------------------------------------------------------------------------|---------------------------------------------------|
| Tracing    | Captures prompt, retrieved context, tool calls, latency & token cost per case | Debug why a summary was wrong                     |
| Evaluation | Faithfulness / hallucination / relevance evaluators on a curated case set     | A hallucinated fraud rationale is worse than none |
| Monitoring | Analyst thumbs up/down captured per summary; tracks quality drift             | Closes an LLM-specific feedback loop              |
| Cost       | Token spend per chain                                                         | Pairs with the cluster cost dashboard             |

The symmetry to pitch

Two parallel observability stacks: MLflow watches the fraud-scoring model on the hot path; LangSmith watches the LLM analyst assistant. Together they form a mature MLOps + LLMOps narrative.

## 07B Fine-Tuned Analyst LLM (Feedback Loop)

The analyst-assist layer is not static. Every summary the analyst rates with a thumbs-up/down is captured, and those judgements are used to fine-tune a dedicated analyst LLM. Crucially, LangChain is retained as the orchestration layer — the fine-tuned model is swapped in behind the same summary / RAG / agent chains, so the rest of the system is unchanged.

### The closed-loop pipeline

- **Capture** — The dashboard persists each thumbs-up / thumbs-down with the prompt context (transaction, SHAP features, retrieved cases) locally and to LangSmith.
- **Curate** — Thumbs-up, grounded, cited summaries are exported to a chat-format JSONL dataset once a minimum example threshold is reached.
- **Fine-tune** — A managed fine-tune (OpenAI) runs when a key is present; otherwise a local LoRA / PEFT adapter is trained on a small base model — no cloud required.
- **Serve** — Setting `LLM_PROVIDER=finetuned` routes the LangChain chains to the fine-tuned model (hosted ft: id or local adapter) via one provider seam.
- **Evaluate** — LangSmith faithfulness / relevance / hallucination evaluators gate the fine-tuned model before it replaces the incumbent analyst LLM.

### Provider abstraction (one seam, four modes)

| Mode                      | Model served              | When it is used                         |
|---------------------------|---------------------------|-----------------------------------------|
| <b>fake</b>               | Deterministic FakeListLLM | No keys — offline demos & unit tests    |
| <b>openai / anthropic</b> | Hosted foundation model   | API key present; base analyst quality   |
| <b>finetuned (hosted)</b> | ft:gpt-... fine-tune      | After a managed fine-tune job succeeds  |
| <b>finetuned (local)</b>  | Base + LoRA adapter       | Air-gapped / no-cost fine-tune fallback |

#### Design principle #4 — keep the LLM swappable and off the hot path

LangChain stays the stable orchestration seam; the model behind it evolves from foundation → fine-tuned as analyst feedback accumulates. The fine-tuned model still only runs on demand when an analyst clicks “Explain”, never inline in scoring.

09

Data Engineering Backbone

Just enough data-engineering tooling to signal production-readiness without over-scoping the project.

| Tool                               | Role in this project                                                           | Why it matters                                        |
|------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------|
| Delta Lake (medallion)             | bronze = raw Kafka dump · silver = cleaned/enriched · gold = training features | Lineage, audit trail, reproducible retraining         |
| Databricks Workflows / Airflow     | Orchestrate periodic retraining & evaluation                                   | Separates streaming (hot) from scheduled batch (cold) |
| Feature Store (Databricks / Feast) | Same feature definitions online & offline                                      | Eliminates training/serving skew                      |
| Auto Loader                        | Incremental ingest of archived/batch files                                     | Backfills without custom code                         |
| Great Expectations                 | Data-quality gate on the silver layer                                          | Stops garbage features poisoning the model            |

Keep it a hint, not a pivot

For a lightweight footprint, Delta Lake + one orchestrator + the Feature Store is enough to prove you understand demo-vs-production — without ballooning scope.

10

End-to-End Data Flow

The eight-second story, at a glance:



| Step | What happens                                                                                                                                                                                                      |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Producer emits a transaction to Kafka topic raw-transactions.                                                                                                                                                     |
| 2    | Databricks stream reads it, updates the user's rolling-window state, and builds the feature vector (incl. graph features from Neptune).                                                                           |
| 3    | The stream loads the MLflow Production model, scores the transaction, and assigns a risk band.                                                                                                                    |
| 4    | Decision written to MongoDB (flagged_transactions + user_rolling_state upsert); raw + scored row appended to Delta bronze/silver.                                                                                 |
| 5    | Streamlit auto-refreshes and shows the new alert on the feed and geo-map.                                                                                                                                         |
| 6    | Analyst clicks "Explain this case" → LangChain summary + RAG chain pulls SHAP features, history, and similar past cases (Mongo Vector Search) to generate a grounded narrative; optional agent queries the graph. |
| 7    | LangSmith traces the whole run — prompt, context, tool calls, latency, tokens — and evaluates faithfulness.                                                                                                       |
| 8    | Analyst labels the case and rates the summary → labels feed MLflow retraining; summary feedback grows the LangSmith eval set. Better models are promoted with no stream redeploy.                                 |

## 11

## Data Models & Schemas

### Kafka event payload (raw-transactions)

```
{
  "txn_id": "b8f1...e2",      "user_id": "U_10432",
  "amount": 1299.00,         "currency": "INR",
  "merchant": "QuickCart",   "merchant_category": "electronics",
  "geo": { "lat": 19.07, "lon": 72.87, "country": "IN" },
  "device_id": "D_88",       "ip": "203.0.113.7",
  "timestamp": "2026-07-25T04:51:22Z"
}
```

### MongoDB — flagged\_transactions document

```
{
  "_id": "b8f1...e2",          // = txn_id (idempotent)
  "user_id": "U_10432",
  "amount": 1299.00,
  "risk_score": 0.87,          "risk_band": "high",
  "features": { "amt_z": 8.1, "txn_1m": 4, "new_geo": true },
  "graph": { "ring_id": "R_22", "shared_device_flags": 3 },
  "shap_top": [ ["amt_z", 0.41], ["txn_1m", 0.28] ],
  "status": "open",           "created_at": "2026-07-25T04:51:23Z"
}
```

## 12

## Trade-offs & Design Decisions

| Decision                   | Trade-off / rationale                                                                                                        |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Stateful streaming windows | State grows with active users → bound it with watermarks + state TTL.                                                        |
| Imbalanced learning        | Optimise recall on fraud; accept some false positives but manage analyst fatigue.                                            |
| Hot vs cold consistency    | MongoDB is eventually consistent with Delta — acceptable: Delta is truth for training, Mongo for serving.                    |
| LLM off the hot path       | Adds a manual step, but protects the sub-second scoring SLA and controls token cost.                                         |
| MongoDB as serving store   | Chosen for document-shaped enriched records + fast reads + optional vector search — defensible over a relational store here. |
| Cost of always-on stream   | Streaming clusters run continuously → use small autoscaling job clusters; single-node for the demo.                          |

## 13

## Security & Reliability

### Security & data integrity

- **PII protection:** tokenize/encrypt sensitive fields (PAN, card number) with MongoDB Queryable Encryption or field-level encryption.
- **Input validation** on the Kafka ingress to reject malformed / poisoned events — guards the retraining set against data-poisoning.
- **Adversarial testing:** simulate fraudsters probing thresholds (slow-ramp amounts) and show the system detecting the pattern, not just the point.

### Reliability & scale

- **Dead-letter queue** in Kafka for un-processable events.
- **Backpressure & autoscaling** for the Databricks stream under burst load.
- **Chaos test:** kill a consumer mid-stream and demonstrate checkpoint recovery with zero data loss (exactly-once proof).

#### Demo-winning polish

A live "inject a card-testing burst now" button in Streamlit, plus a cost dashboard (events processed vs cluster spend) and a p50/p95/p99 latency panel, make the demo gripping and quantified.

14

Build Plan & Milestones

Phased four-week plan

| Phase  | Focus                  | Deliverable                                                                                                             |
|--------|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Week 1 | Core plumbing          | Kafka + synthetic producer → Databricks stream → Mongo + Delta; dummy scorer; basic Streamlit feed                      |
| Week 2 | Real ML + graph        | Rolling features, Isolation Forest via MLflow, Feature Store, Neptune graph + ring detection, SHAP in review panel      |
| Week 3 | LLM assistant          | LangChain summary + RAG over Mongo Vector Search; wire the "Explain" button; add the investigation agent if time allows |
| Week 4 | Observability & polish | LangSmith tracing + eval + feedback; drift detection + auto-retrain; cost/SLA dashboards; live attack-injection demo    |

Compressed 2-week option

Fold the LangChain summary chain (no agent) into Week 2, and LangSmith tracing (no formal eval) into a stretch task. You still have a demo-able LLM-explainability story — matching an aggressive-but-realistic timeline.

15

Tech Stack & Success Metrics

Full technology stack

| Layer                          | Technology                                                              |
|--------------------------------|-------------------------------------------------------------------------|
| Ingestion / streaming bus      | Apache Kafka (Kafka Connect, Debezium optional)                         |
| Stream processing / ML compute | Databricks Structured Streaming (PySpark)                               |
| ML lifecycle                   | MLflow (tracking + registry), Isolation Forest → XGBoost/LightGBM, SHAP |
| Graph                          | AWS Neptune or Neo4j (Gremlin / openCypher)                             |
| Serving store                  | MongoDB (+ Atlas Vector Search)                                         |
| LLM orchestration              | LangChain (chains, RAG, agents)                                         |
| LLM observability              | LangSmith (tracing, evaluation, monitoring)                             |
| Storage / table format         | Delta Lake (medallion) — Iceberg optional                               |
| Orchestration                  | Databricks Workflows or Airflow / Dagster                               |
| Data quality                   | Great Expectations / Soda                                               |
| UI / dashboard                 | Streamlit                                                               |

Success metrics (put these in your README)

| Metric                     | What good looks like                                                     |
|----------------------------|--------------------------------------------------------------------------|
| End-to-end scoring latency | p95 < 2 s under target load                                              |
| Throughput                 | Sustains target events/sec without lag growth                            |
| Detection quality          | Precision/recall on fraud class + expected-loss captured                 |
| Explanation faithfulness   | LangSmith faithfulness score above threshold; low hallucination rate     |
| Adaptability               | Retrain triggered on drift; challenger beats production before promotion |
| Cost efficiency            | \$ per million events processed, trended over time                       |

**Bottom line:** this specification describes a system that is real-time, explainable, graph-aware, and observably self-improving — a portfolio piece that reads like production engineering, not a demo.